

PROJECT: AWS ECS Fargate deployment with Terraform, CodePipeline CI/CD, and CloudWatch monitoring

In this project, I would like to tell u the basics of FLASK API, Terraform ,AWS CLI, and some AWS services in a very simple way but with deep explanation and practical manner and i made this project so that it is like production level. I want students like me to go through this project so that they get to know the basics, they get to know how things work at enterprise level, and also to get jobs in devops with terraform, aws services. I would like u to add some points so that others feel that they can learn something valuable by looking at this project since I explained a superb project like this with basics defined. My main motto is not to showcase my projects, i want others to grasp the fundamentals of the topics. MAInly there are IP NETworking fundamentals here liek Ip addresses, CIDR and all that that can help u in cybersecurity also.

Github link of this project: <https://github.com/CharithKapuluru/proj2-ecs-fargate>

I divided this project into phases.

PHASE: A - Create Flask Application and Docker Image

Goal:

Create a simple web application and package it into a Docker container that can run anywhere - on your laptop, in AWS, or any cloud provider.

What We're Building:

A minimal Flask web application with:

- Homepage endpoint (/) that returns a JSON greeting
- Health check endpoint (/health) for AWS to monitor the app
- Request logging with timing data (observability)

Command:

```
mkdir -p proj2-ecs-fargate/app
mkdir -p proj2-ecs-fargate/terraform
cd proj2-ecs-fargate
```

What This Does:

Creates the directory structure for our project.

Why This Structure?

Separation of concerns:

- app/ = Application code (what the app does)
- terraform/ = Infrastructure code (how to deploy it)

This is a professional standard - keeps code organized and maintainable.

Firstly, I would like to tell you the difference between REST API and FLASK API.

Imagine you are building the backend for a **food delivery app** (like Uber Eats).

1 REST API = The Rulebook (First)

REST says:

Action	REST Rule
View all restaurants	GET /restaurants
View one restaurant	GET /restaurants/12
Add a restaurant	POST /restaurants
Update a restaurant	PUT /restaurants/12
Delete a restaurant	DELETE /restaurants/12

📌 These rules **exist before Flask or FastAPI**.

They are **framework-independent**.

2 What Happens When a Request Comes In (Common Flow)

This flow is **IDENTICAL** for Flask and FastAPI:

Client → Request → Framework → Python Logic → Response → Client

Now let's walk through one request.

3 Example: GET /restaurants/12

Step A: Client Sends Request

A mobile app or browser sends:

- Method: GET
- URL: /restaurants/12

Meaning (REST rule):

“Give me restaurant with ID = 12”

4 Flask's Role (Executor)

When **Flask** receives this request:

1. Flask **listens** for incoming HTTP requests
2. It checks:
 - Path: /restaurants/12
 - Method: GET
3. Flask finds:
 - “Ah, this matches the rule for fetching a restaurant”
4. Flask runs **Python logic** that:
 - Reads 12
 - Fetches restaurant data from database
5. Flask **wraps the result** into:
 - JSON response

6. Flask sends it back to the client

📦 Flask does **not** invent rules

📦 Flask **executes** REST rules

5 FastAPI's Role (Same Job, Faster & Smarter)

Now imagine the **same REST request** hits **FastAPI** instead.

FastAPI does the **same steps**, but with extras:

1. Receives **GET /restaurants/12**
2. Validates:
 - Is **12** actually a number?
3. Automatically:
 - Documents the endpoint
 - Checks response format
4. Runs Python logic
5. Sends JSON response

📦 REST rules are the same

📦 FastAPI just adds **automation and speed**

6 Key Insight (Very Important)

REST does not care which framework you use.

This is huge.

Layer	Responsibility
REST	Rules & structure
Flask	Executes rules manually
FastAPI	Executes rules with automation

7 Another Quick Example: Creating an Order

REST Rule:

POST /orders

Meaning:

“Create a new order”

What Flask Does:

- Receives order data
- Runs Python logic
- Stores it
- Returns confirmation

What FastAPI Does:

- Receives order data
- Validates it automatically
- Runs Python logic
- Stores it
- Returns confirmation

Same rulebook.

Different executors.

Terraform:

What is Infrastructure as Code (IaC)?

Infrastructure as Code (IaC) means:

You define your cloud infrastructure using code, and a tool automatically creates, updates, or deletes that infrastructure for you.

So instead of:

- Clicking buttons in the AWS Console
- Manually creating VPCs, subnets, security groups, etc.

You:

- **Write code**
- **Run a command**
- **Infrastructure appears exactly as described**

Terraform is a tool that lets you **create, change, and manage infrastructure using code** instead of manual clicks.

In one sentence:

Terraform allows you to define your cloud infrastructure in code and automatically provision it across platforms like AWS, Azure, and GCP.

Terraform is built by **HashiCorp** and is one of the **most widely used IaC tools in real companies**.

=====

DOCKER COMMANDS

=====

BUILD IMAGE:

```
docker build -t proj2-app:local ./app
```

-t proj2-app:local Tag the image with name:version

./app Build from ./app directory (where Dockerfile is)

RUN CONTAINER:

```
docker run -d -p 8080:8080 --name proj2-test proj2-app:local
```

-d Run in background (detached)
-p 8080:8080 Map host port 8080 to container port 8080
--name proj2-test Name the container

TEST ENDPOINTS:

```
curl http://localhost:8080/  
# {"message":"Hello from ECS Fargate!","service":"proj2-app"}
```

```
curl http://localhost:8080/health  
# {"status":"ok","timestamp":"2026-01-16T21:20:18.962999"}
```

VIEW LOGS:

```
docker logs proj2-test  
  
# Shows unicorn startup + request logs:  
# GET / - 200 - 0.6ms  
# GET /health - 200 - 0.1ms
```

CLEANUP:

```
docker stop proj2-test  
docker rm proj2-test
```

OTHER USEFUL COMMANDS:

```
docker ps                # List running containers
```

docker images | grep proj2 # List images

docker logs -f proj2-test # Follow logs in real-time

PHASE B: AWS CLI SETUP + IAM + BUDGET ALERT

In Phase B, we verified our AWS setup before creating any resources. We confirmed that the AWS CLI was properly configured and

authenticated as the test_user IAM user by running `aws sts get-caller-identity`. We verified the default region was set to us-east-1

(the cheapest and most feature-complete AWS region). We created a budget alert called proj2-cost-guardrail with a \$10/month limit

that sends an email notification when spending exceeds 80% (\$8) — this acts as a safety net to prevent surprise bills. Finally, we

tested that we had the necessary permissions to access ECR, ECS, and VPC services by running `describe/list` commands and confirming

no "Access Denied" errors. This phase was all about preparation and safety checks — no infrastructure was built yet, just making

sure we had the keys, knew where we were going, and had cost protection in place.

The screenshot displays the AWS Billing and Cost Management console. The left sidebar shows the navigation menu with categories like Billing and Payments, Cost and Usage Analysis, and Cost Organization. The main content area shows the details for a budget named 'proj2-cost-guardrail'. The 'Budget health' section shows 'Current vs. budgeted' at 23.96% and 'Forecasted vs budgeted (MTD)' at 0.00%. The 'Alerts' section shows 'Thresholds' as 'OK' and 'Actions' as '-'. The 'Details' section shows 'Health status' as 'Healthy', 'Budget amount' as '\$10.00', 'Budget type' as 'Cost budget', 'Period' as 'Monthly', 'Start date' as '2026-01-01', and 'End date' as '-'. The 'Additional budget parameters' section is currently collapsed. The 'Budget history' section is visible at the bottom, with a 'View in AWS Cost Explorer' link.

Budget health	
Current vs. budgeted	Forecasted vs budgeted (MTD)
Amount spent: \$2.40 of \$10.00	Amount spent: - of \$10.00

Details		
Health status	Budget amount	Start date
Healthy	\$10.00	2026-01-01
Budget type	Period	End date
Cost budget	Monthly	-

Budget history	
View in AWS Cost Explorer	

What is AWS CLI? (Simple)

AWS CLI (Command Line Interface) is a tool that lets you **talk to AWS services by typing commands** instead of clicking in the AWS Console.

List your S3 buckets

```
bash
```

[Copy code](#)

```
aws s3 ls
```

This does the same thing as:

AWS Console → S3 → View buckets

Example with EC2

List EC2 instances

```
bash
```

[Copy code](#)

```
aws ec2 describe-instances
```

Stop an EC2 instance

```
bash
```

[Copy code](#)

```
aws ec2 stop-instances --instance-ids i-0123456789abcdef
```

Example with ECS (your context)

List ECS clusters

```
bash
```

[Copy code](#)

```
aws ecs list-clusters
```

VPC:

What is AWS VPC? (Simple)

An **AWS VPC (Virtual Private Cloud)** is **your own private network inside AWS** where you control:

- IP addresses
- Subnets
- Routing
- Security

Think of it as **your own isolated data-center network in the cloud**.

Why VPC exists

AWS doesn't just drop servers on the internet.

A VPC lets you decide:

- Who can access what
 - Which resources are public vs private
 - How traffic flows
-

Simple example (realistic)

You create a VPC:

```
nginx
```

[Copy code](#)

```
VPC CIDR: 10.0.0.0/16
```

Inside it:

- **Public subnet:** 10.0.1.0/24 (Load Balancer)
- **Private subnet:** 10.0.101.0/24 (App, DB)

If you create a VPC using **any IAM user** under the **same AWS root account**, that VPC belongs to the **AWS account**, not the IAM user — and **other IAM users can access it *if* they have permission**.

Key Rule (MEMORIZE THIS)

AWS resources belong to the AWS account, not to IAM users.

IAM users are just:

- Identities
- Permission holders

They **do not own infrastructure**.

What Actually Happens When You Create a VPC

Let's say:

- Root AWS account: **Company-Prod-Account**
- IAM User A creates a VPC
- IAM User B logs in later

The VPC is:

- ✓ Inside the **same AWS account**
 - ✓ Stored at the **account level**
 - ✓ Visible to **any IAM identity with permission**
-

Why Some Users “Can’t See” the VPC

This is where people get confused.

 **It's NOT because:**

- Different IAM users
- Different logins

 **It IS because:**

- **IAM policies**

- **Region mismatch**
-

IAM Permissions Control Visibility

If IAM User B has:

```
{  
  "Effect": "Allow",  
  "Action": "ec2:DescribeVpcs",  
  "Resource": "*" }  
}
```

Then:

- They **can see** the VPC
- They **can use** it
- They **can attach resources to it** (if allowed)

If they **don't** have permissions:

- The VPC still exists
 - They just can't see or modify it
-

Region Matters (Very Common Mistake)

VPCs are **regional**.

Example:

- VPC created in `us-east-1`

- User checks `us-west-2`

→ “VPC is missing” ❌

→ Actually, wrong region ✅

What is a VPC?

A **VPC (Virtual Private Cloud)** is **your own private network inside AWS**.

Think of it like:

A **virtual data center network** where you control:

- IP address ranges
- Subnets
- Routing
- Security

A **subnet** is:

A **smaller network carved out of a VPC's IP range**

If:

- VPC = big land
- Subnet = plots inside that land

You **must** create subnets to place:

- EC2
- Load balancers
- RDS
- ECS tasks

Nothing runs directly in a VPC — **everything runs inside a subnet.**

Public Subnet (Simple)

A **public subnet** is a subnet that **can reach the internet directly.**

Why?

Because its route table has:

```
java
0.0.0.0/0 → Internet Gateway (IGW)
```

 Copy code

Typical use

- Load Balancer (ALB)
- NAT Gateway
- Bastion host

Example

```
yaml
Public Subnet: 10.0.1.0/24
Resource: ALB
Public IP: Yes
```

 Copy code

Users can access it from the internet.

Private Subnet (Simple)

A **private subnet** is a subnet that **cannot be reached directly from the internet**.

Why?

Because it has **no route to IGW**.

It may have:

```
0.0.0.0/0 → NAT Gateway
```

 Copy code

(for outbound only)

Typical use

- App servers
- ECS tasks
- Databases (RDS)

Example

```
yaml
```

```
Private Subnet: 10.0.101.0/24
Resource: App server
Public IP: No
```

 Copy code

Users cannot reach it directly.

Real production example (most common)

VPC: 10.0.0.0/16

Public subnets (in 2 AZs)

- 10.0.1.0/24 (AZ-a)
- 10.0.2.0/24 (AZ-b)

Private subnets (in 2 AZs)

- 10.0.101.0/24 (AZ-a)
- 10.0.102.0/24 (AZ-b)

Now place components:

✓ Public subnet contains

1) Internet-facing Load Balancer (ALB)

Why public?

Because users on the internet must reach it.

Flow:

- User → Internet → **ALB public endpoint** → forwards to app in private subnet

ALB is basically the **front door** to your app.

2) NAT Gateway (usually)

Why public?

Because NAT needs to talk to the internet using an IGW (it has an Elastic IP).

Private subnets use NAT for outbound.

✅ Private subnet contains

3) App servers / ECS tasks (backend)

Why private?

Because you *don't want the internet directly hitting your servers*.

Instead:

- Only ALB can send traffic to app servers (via Security Group rule: allow from ALB SG)

Flow:

- ALB → App (private IP) ✅
- Internet → App ❌ (blocked)

This reduces:

- hacking surface
- port scanning exposure
- accidental public access

4) Database (RDS)

Why private?

Because databases should **never** be exposed to the internet.

Flow:

- App → DB ✅ (private IP traffic inside VPC)
- Internet → DB ❌

Most security incidents happen when DB is public by mistake.

5 NAT Gateway sends traffic to:

```
java
```

[Copy code](#)

```
Internet Gateway (IGW)
```

6 IGW sends it to the internet

7 Response comes back:

```
nginx
```

[Copy code](#)

```
Internet → IGW → NAT Gateway → 10.0.101.10
```

✓ Outbound works

✗ Inbound from internet still blocked

🧠 Why NAT Gateway MUST be in a Public Subnet

Because:

- NAT Gateway needs a **public IP**
- Only public subnets have route:

```
0.0.0.0/0 → IGW
```

[Copy code](#)

So NAT Gateway can:

- Reach the internet
 - Return responses
-

Detailed traffic story (this makes it click)

Scenario: User opens your website

1. User types `https://myapp.com`
2. DNS points to ALB
3. User traffic hits ALB (public subnet)
4. ALB forwards to an ECS task / EC2 app server (private subnet)
5. App server queries database (private subnet)
6. Response goes back:
App → ALB → User

At no point does the user talk directly to app or DB.

Short answer first (important)

No. An Internet Gateway (IGW) does NOT stay in a public subnet.

Correct understanding

An Internet Gateway is attached to the VPC, not to any subnet.

Subnets (public or private) **do not contain** the IGW.

1 What is a CIDR block? (Very simple)

A CIDR block defines a range of IP addresses.

Example:

```
10.0.0.0/16
```

 Copy code

Means:

"I am reserving a block of IP addresses starting from 10.0.0.0."

CIDR answers **one question**:

"How many IP addresses do I have to use?"

2 VPC CIDR (Big Network)

Example VPC

```
nginx
```

```
VPC CIDR: 10.0.0.0/16
```

 Copy code

This means:

- Total IPs: **65,536**
- Range:

```
10.0.0.0 → 10.0.255.255
```

 Copy code

Think of the VPC as:

 A big city with a lot of addresses



Everything inside this city:

- EC2
- ECS
- ALB
- RDS
- NAT Gateway

👉 All must use IPs from this range

3 Why /16 for VPC?

/16 means:

- First **16 bits** identify the VPC (network)
- Remaining **16 bits** are available inside

Practically:

- Large enough to create many subnets
- Industry standard
- Easy to manage

Companies almost always use:

- /16 or /20 for VPC
-

4 Subnets = smaller CIDR blocks inside VPC

Now we **split the VPC CIDR** into subnets.

Important rule:

Subnet CIDRs must be inside the VPC CIDR and must NOT overlap.

5 Public Subnet CIDR (Example)

nginx

 Copy code

Public Subnet 1: 10.0.1.0/24

Public Subnet 2: 10.0.2.0/24

Each /24 :

- Total IPs: 256
- Usable IPs in AWS: 251

Range:

10.0.1.0 → 10.0.1.255

10.0.2.0 → 10.0.2.255

 Copy code

These are **small neighborhoods** inside the city.

6 Private Subnet CIDR (Example)

nginx

 Copy code

Private Subnet 1: 10.0.101.0/24

Private Subnet 2: 10.0.102.0/24

Range:

10.0.101.0 → 10.0.101.255

10.0.102.0 → 10.0.102.255

 Copy code

Notice:

- Still inside 10.0.0.0/16
- Clearly separated



8 What lives in each subnet (with IP examples)

Public subnet (10.0.1.0/24)

- ALB → 10.0.1.10
- NAT Gateway → 10.0.1.20

Why public?

- Needs internet access
- Route table:

 Copy code

0.0.0.0/0 → IGW

Private subnet (10.0.101.0/24)

- ECS task → 10.0.101.10
- App EC2 → 10.0.101.20
- RDS → 10.0.101.30

Why private?

- Should NOT be accessed directly from internet
- Route table:

 Copy code

0.0.0.0/0 → NAT Gateway

9 Full traffic flow using CIDRs (real story)

User opens your app

nginx

 Copy code

User → ALB (10.0.1.10)

ALB forwards to:

nginx

 Copy code

ALB → ECS task (10.0.101.10)

ECS talks to DB:

10.0.101.10 → 10.0.101.30

 Copy code

All private IPs, all inside the VPC CIDR.

ECS task needs internet (outbound)

SCSS

 Copy code

10.0.101.10 → NAT (10.0.1.20) → IGW → Internet

Private subnet never touches IGW directly.

PHASE C: ECR REPOSITORY + PUSH IMAGE - DETAILED EXPLANATION

Goal:

Upload your Docker image from your Mac to AWS ECR so that ECS can later download it and run it in the cloud.

What We're Doing:

1. Create a "repository" in ECR (like creating an album in iCloud)
2. Log Docker into ECR (authenticate)
3. Tag image with ECR address (rename for upload)
4. Push image to ECR (upload)

5. Verify it uploaded correctly

After This Phase:

Your Docker image will be:

- ✓ Stored in AWS
- ✓ Accessible to ECS
- ✓ Scanned for vulnerabilities
- ✓ Encrypted at rest
- ✓ Ready for deployment

1 ECR — Amazon Elastic Container Registry

What it is (simple):

ECR is AWS's Docker image storage.

It stores your built container images securely.

Think: ECR = Docker Hub, but private and inside AWS.

What goes into ECR?

- Docker images (`myapp:latest` , `myapp:v1`)
- Image layers + metadata

What ECR does NOT do

- It does **not** run containers
- It only **stores** images

Example

text

 Copy code

```
ECR repository: myapp-repo  
Image tags: myapp:v1, myapp:v2
```

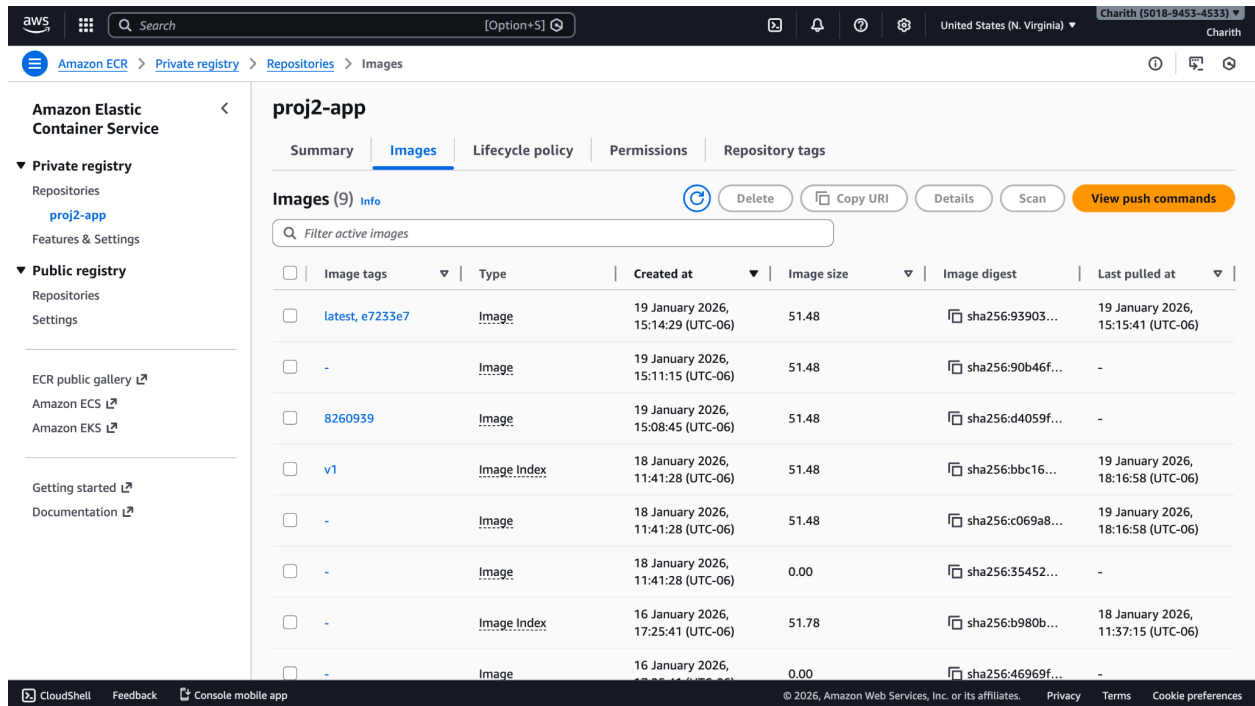
You build locally or in CI:

bash

 Copy code

```
docker build -t myapp:v1 .  
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/myapp-repo:v1
```

Now the image lives in ECR.



PHASE D: TERRAFORM VPC + ALB INFRASTRUCTURE - DETAILED EXPLANATION

In Phase D, we used Terraform to provision the core AWS networking infrastructure as code. Instead of manually clicking through the

AWS Console, we wrote Terraform configuration files (.tf files) that define our desired infrastructure declaratively. We created a

VPC with CIDR block 10.0.0.0/16, two public subnets across different availability zones (us-east-1a and us-east-1b) for high

availability, an Internet Gateway to allow internet access, route tables to direct traffic, and two security groups — one for the

ALB (allowing HTTP port 80 from the internet) and one for ECS tasks (allowing traffic only from the ALB). We also set up an

Application Load Balancer (ALB) with a target group and HTTP listener. Running terraform init, terraform plan, and terraform apply

created all 12 resources automatically. At this point, the ALB returns a 503 error because there are no targets yet — the ECS tasks

haven't been deployed. This phase established the network foundation that our containers will run on.

What is an ALB? (Simple)

An Application Load Balancer (ALB) is a service that:

Receives HTTP/HTTPS traffic from users and intelligently distributes it to backend services (EC2/ECS tasks) based on rules.

Terraform commands:

🔴 terraform init

Downloads and installs the required provider plugins (like the AWS provider) and sets up the working directory. Think of it like npm

install — it fetches the dependencies Terraform needs to talk to AWS. You run this once when starting a project or when you add new providers.

terraform plan

Shows you a preview of what Terraform will do without actually doing it. It compares your .tf files (what you want) against the

current state (what exists) and outputs a diff: resources to create (+), modify (~), or destroy (-). This is your safety check — review the plan before applying.

terraform apply

Actually executes the changes. It creates, modifies, or destroys resources in AWS to match what's defined in your .tf files. After

running, it updates the state file (terraform.tfstate) to track what exists. You'll be prompted to type yes to confirm (or use -auto-approve to skip).

- terraform destroy — Tears down all resources defined in your .tf files**
- terraform validate — Checks syntax without connecting to AWS**
- terraform fmt — Auto-formats your .tf files**

PHASE E: ECS FARGATE DEPLOYMENT

In Phase E, we deployed our containerized Flask application to AWS ECS Fargate. We created an IAM role (`ecsTaskExecutionRole`) that gives ECS permission to pull Docker images from ECR and write logs to CloudWatch. We set up a CloudWatch Log Group (`/ecs/proj2-app`) to capture container output. We then created an ECS Cluster (`proj2-cluster`) as a logical grouping for our containers, and a Task Definition that specifies exactly how to run our container — the Docker image from ECR, CPU (256 units = 0.25 vCPU), memory (512MB), port mapping (8080), and logging configuration. Finally, we created an ECS Service that maintains our desired task count, connects to the ALB target group, and automatically replaces unhealthy tasks. After terraform apply, the ALB health checks pass, and hitting the ALB URL now returns `{"message": "Hello from ECS Fargate!", "service": "proj2-app"}` — our app is live. The complete request flow is: User → ALB (port 80) → ECS Task (port 8080) → Flask app → response back through the same path.

2 ECS — Amazon Elastic Container Service

What it is (simple):

ECS is AWS's container runner and manager.

It pulls images from ECR and runs them as containers.

Think: ECS = the service that starts, stops, restarts, and scales containers.

Key ECS concepts (quick)

- **Task Definition:** blueprint (which image, CPU, memory, port)
- **Task:** a running container
- **Service:** keeps N tasks running (self-healing)
- **Cluster:** logical place where services/tasks live
- **Compute:** where tasks run → EC2 or Fargate

3 How ECR and ECS work together (the flow)

One realistic example: Web API on ECS Fargate

You want: a web API that always has 2 running containers behind an ALB.

Step A — Build & store image (ECR)

- Build Docker image for your API
- Push to ECR

text

 Copy code

```
ECR: myapp-repo: v1
```

Step B — Tell ECS how to run it (Task Definition)

You create a **Task Definition**:

text

 Copy code

```
Image: ECR/myapp-repo:v1
CPU: 256
Memory: 512
Port: 3000
Logs: CloudWatch
```

This is just instructions.

Step C — Run it with ECS (Service)

You create an **ECS Service**:

text

 Copy code

```
Desired tasks: 2
Launch type: Fargate
Attach to ALB target group
```

ECS now:

- Pulls the image from **ECR**
- Starts **2 tasks** (containers)
- Gives each task a **private IP**
- Registers them with the ALB

If a task crashes:

- ECS **automatically starts a new one**
-

The Core Question You're Asking (Rephrased)

You're basically asking:

"If ECS runs Docker containers, **why do EC2 instances or Fargate matter at all?**
Why can't ECS just 'run containers' by itself?"

This is the **right question**.

The Missing Concept: Containers Need a Place to Run

A **Docker container cannot run in the air**.

It always needs:

- CPU
- Memory
- Network
- Disk

Those come from **some machine**.

So the real question ECS must answer is:

“WHERE should your containers actually run?”

That’s where **EC2** and **Fargate** come in.

Think in Layers (Very Important)

SCSS

 Copy code

Your App (Docker Container)

↓

ECS (decides what to run, how many, when)

↓

Compute (where it actually runs)

├─ EC2

└─ Fargate

👉 ECS = manager / orchestrator

👉 EC2 or Fargate = the actual place containers run

Option 1: ECS on EC2 (You manage the machines)

What this means (plain English)

You provide EC2 instances.
ECS places containers on those EC2 instances.

Real example

You do this:

1. Launch 3 EC2 instances
2. Install ECS agent on them
3. Create an ECS cluster
4. Run tasks

Now:

- Each EC2 instance is like a **host**
- Containers run *inside* those EC2s

arduino

 Copy code

EC2 instance

├─ Task 1 (container)
├─ Task 2 (container)
└─ Task 3 (container)



What YOU are responsible for (important)

With ECS on EC2, you manage:

- EC2 instance size
 - OS updates
 - Patching
 - Scaling EC2 up/down
 - Disk space
 - Failures of EC2
-

When companies choose ECS on EC2

- ✓ Long-running workloads
- ✓ Need GPUs or special hardware
- ✓ Want maximum control
- ✓ Predictable workloads
- ✗ More operational work

☁️ Option 2: ECS on Fargate (Serverless containers)

Now the magic part.

What Fargate really is

Fargate is AWS saying:
"Don't think about servers at all."

You tell ECS:

- "I want 1 container"
- "Give it 0.5 CPU and 1 GB RAM"

AWS:

- Picks machines
- Runs the container
- Handles scaling
- Handles failures

You never see:

- EC2
- OS
- Host machines



Real example

You do this:

yaml

 Copy code

```
Run task:
- Image: myapp
- CPU: 256
- Memory: 512
```

AWS does:

css

 Copy code

```
Invisible machine
└─ Your task (container)
```

If it crashes:

- AWS restarts it
- You don't care where

What YOU manage with Fargate

- ✓ Docker image
- ✓ CPU & memory settings
- ✓ Networking
- ✗ Servers
- ✗ OS
- ✗ Scaling infrastructure

When companies choose Fargate

- ✓ Web apps
- ✓ APIs
- ✓ Microservices
- ✓ Beginners
- ✓ Most modern architectures
- ✗ Need special hardware

Because companies have **different needs**.

Need	EC2	Fargate	
Control over servers	✓	✗	
No server management	✗	✓	
Simple setup	✗	✓	
Cost predictability	✓	⚠	
Beginner friendly	✗	✓	

Very Important Clarification (This unlocks everything)

ECS itself does NOT run containers.
ECS only decides *what* should run and *how many*.

The **actual execution** happens on:

- EC2 (you manage)
- OR Fargate (AWS manages)



An **IAM Role** is:

A set of permissions that an AWS service or resource can temporarily assume to access other AWS services.

aws

Search

[Option+S]

Global

Charith (5018-9453-4533)

Charith

IAM

Roles

proj2-ecs-task-execution-role

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management

Temporary delegation requests

New

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

proj2-ecs-task-execution-role

Info

Delete

Summary

Creation date

January 18, 2026, 11:36 (UTC-06:00)

ARN

arn:aws:iam::501894534533:role/proj2-ecs-task-execution-role

Last activity

9 minutes ago

Maximum session duration

1 hour

Permissions

Trust relationships

Tags (4)

Last Accessed

Revoke sessions

Permissions policies (1)

Info

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Filter by Type

Search

All types

1

Policy name

Type

Attached entities

AmazonECSTaskExecutionRolePolicy

AWS managed

1

Permissions boundary (not set)

Generate policy based on CloudTrail events

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

What is CloudWatch? (Simple)

Amazon Web Services CloudWatch is AWS's service for monitoring, logs, and alerts.

CloudWatch answers 3 questions:

1. *"What is happening right now? (metrics)"*
2. *"What happened? (logs)"*
3. *"Should I be alerted? (alarms)"*

1 CloudWatch Metrics (numbers over time)

What they are:

Automatic measurements AWS collects every few seconds/minutes.

Examples you'll actually see:

- EC2: CPUUtilization, NetworkIn/Out
- ALB: RequestCount, TargetResponseTime, HTTP 5xx
- ECS: CPUUtilization, MemoryUtilization
- RDS: CPU, FreeStorageSpace

Beginner example

- Your ECS service CPU goes above **80%**
- CloudWatch shows a spike
- You know your app is under load

👉 Metrics = graphs & numbers

aws

Search

[Option+S]

United States (N. Virginia)

Charith (5018-9453-4533)

Charith

CloudWatch

Log management

CloudWatch

<

Favorites and recents

▶

Ingestion

Dashboards

▶ Alarms 2 10 0

▶ AI Operations

▶ GenAI Observability

▶ Application Signals (APM) New

▶ Infrastructure Monitoring

▼ Logs

Log Management New

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

▶ Metrics

▶ Network Monitoring

▶ Setup

Log groups (2)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Exact match

< 1 >

☐

Log group

▼

Log class

▼

Anomaly d...

▼

Deleti...

Dat...

▼

Sens...

▼

Retenti...

▼

☐

[/aws/codebuild/proj2-build](#)

Standard

[Configure](#)

Off

-

-

1 week

☐

[/ecs/proj2-app](#)

Standard

[Configure](#)

Off

-

-

1 week

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

aws

Search

[Option+S]

United States (N. Virginia)

Charith (5018-9453-4533)

Charith

Amazon Elastic Container Service

Clusters

proj2-cluster

Services

Amazon Elastic Container Service

<

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

proj2-cluster

Fargate

Last updated 20 January 2026, 14:58 (UTC-6:00)

Actions

Create with Express Mode

Cluster overview

ARN

arn:aws:ecs:us-east-1:50189453:4533:cluster/proj2-cluster

Status

Active

CloudWatch monitoring

Default

Registered container instances

-

Services

Draining

-

Active

1

Tasks

Pending

-

Running

1

Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Services (1)

Info

Last updated 20 January 2026, 14:58 (UTC-6:00)

Manage tags

Update

Delete service

Create

Filter services by value

Filter launch type Any launch type

Filter scheduling strategy Any scheduling strategy

Filter resource management type Any resource management type

< 1 >

☐

Service name

▲

ARN

Status

Schedu...

▼

L...

▼

Task de...

▼

Deployments and tas

☐

[proj2-service](#)

arn:aws:ecs:us-e

Active

REPLICA

-

[proj2-task:1](#)

https://us-east-1.console.aws.amazon.com/ecs/v2/clusters/proj2-cluster/services?region=us-east-1

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

PHASE F: MONITORING & ALERTING

In Phase F, we set up monitoring and alerting to know when something goes wrong with our application. We created an SNS (Simple

Notification Service) topic and subscribed your email to it, so you receive notifications when alarms trigger. We then created 7

CloudWatch Alarms that monitor critical metrics: high CPU utilization (>80%), high memory utilization (>80%), HTTP 5xx errors

(server errors), HTTP 4xx errors (client errors), unhealthy targets in the load balancer, high ALB response time, and low running

task count. When any of these thresholds are breached, you get an email alert. We also built a CloudWatch Dashboard that displays

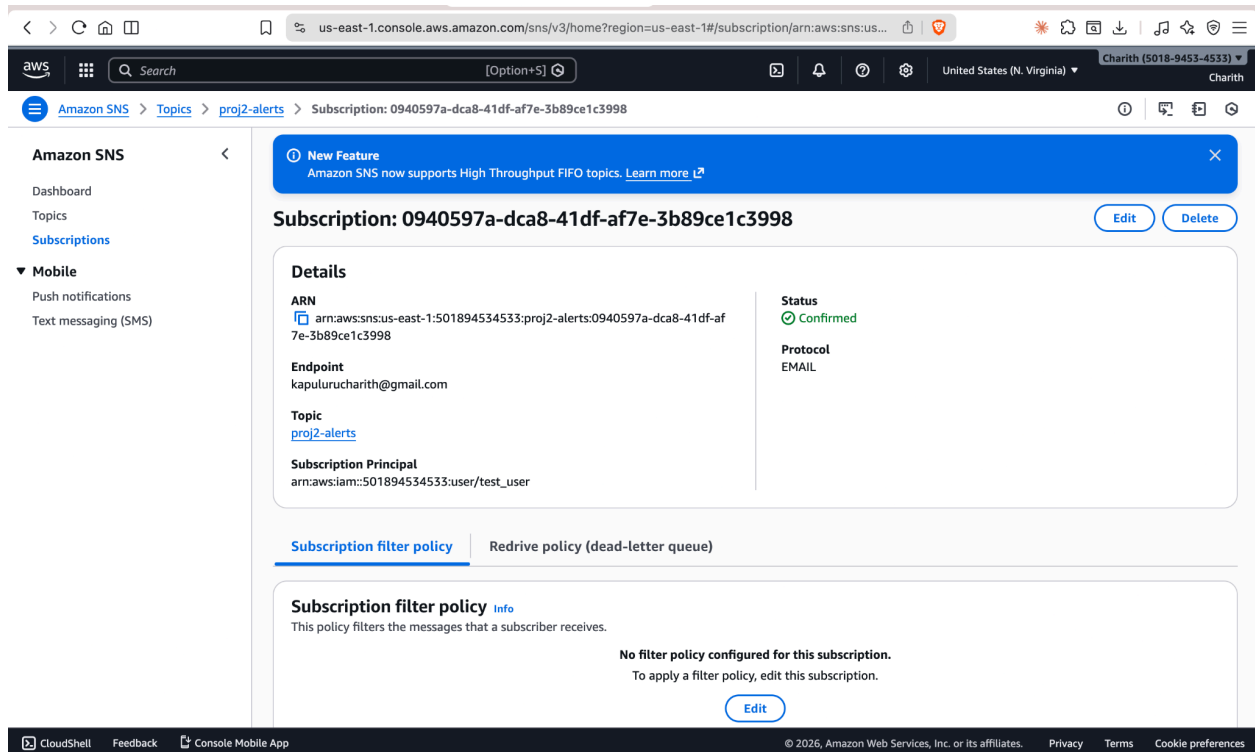
real-time graphs of request count, response times, error rates, CPU/memory usage, healthy host count, and running tasks — giving you

a single pane of glass to monitor your application's health. This transforms the setup from "hope it works" to "know it works" —

you'll be notified of problems before users even notice them.

The screenshot displays the AWS CloudWatch Alarms console. The left sidebar shows navigation options: CloudWatch, Favorites and recents, Ingestion, Dashboards, Alarms (2 active, 10 inactive), In alarm, All alarms, Billing, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, Logs, Log Management, Log Anomalies, Live Tail, Logs Insights, Contributor Insights, and Metrics. The main panel shows a list of 12 alarms. The top of the panel includes filters for 'Hide Auto Scaling alarms', 'Clear selection', 'Create composite alarm', 'Actions', and 'Create alarm'. Below the filters is a table with columns: Name, State, Last state update (UTC), Conditions, and Actions. The table lists four alarms: 1. 'RequestCountPerTarget < 90 for 15 datapoints within 15 minutes' (In alarm), 2. 'CPUUtilization > 70 for 3 datapoints within 3 minutes' (OK), 3. 'CPUUtilization < 63 for 15 datapoints within 15 minutes' (In alarm), and 4. 'RunningTaskCount < 1 for 1 datapoints within 5 minutes' (OK). Each alarm has a link to its configuration and an 'Actions' button. The bottom of the console shows the footer with '© 2026, Amazon Web Services, Inc. or its affiliates.' and links for Privacy, Terms, and Cookie preferences.

Name	State	Last state update (UTC)	Conditions	Actions
TargetTracking-service/proj2-cluster/proj2-service-AlarmLow-1338a296-a39c-4a8a-9779-bd8321f74d0a	In alarm	2026-01-20 15:53:42	RequestCountPerTarget < 90 for 15 datapoints within 15 minutes	Actions enable
TargetTracking-service/proj2-cluster/proj2-service-AlarmHigh-ef8c62c4-8846-469e-8e5d-47f339ac3e45	OK	2026-01-20 00:18:02	CPUUtilization > 70 for 3 datapoints within 3 minutes	Actions enable
TargetTracking-service/proj2-cluster/proj2-service-AlarmLow-44257ed5-1837-444d-9313-927fbb6f1f54	In alarm	2026-01-20 00:17:53	CPUUtilization < 63 for 15 datapoints within 15 minutes	Actions enable
proj2-service-scaled-to-zero	OK	2026-01-20 00:16:51	RunningTaskCount < 1 for 1 datapoints within 5 minutes	No actions



PHASE G: CI/CD PIPELINE WITH AWS CODEPIPELINE

In Phase G, we built an automated CI/CD pipeline using AWS CodePipeline and CodeBuild so that code changes automatically deploy to production. We created a CodeStar Connection to link AWS with your GitHub repository (requires one-time manual authorization in the AWS Console). We set up an S3 bucket to store pipeline artifacts, and created IAM roles with the necessary permissions for CodeBuild and CodePipeline to access ECR, ECS, and other services. We wrote a `buildspec.yml` file that tells CodeBuild exactly what to do: log into ECR, build the Docker image from the `app/` directory, tag it with the commit hash, push it to ECR, and generate an `imagedefinitions.json` file that tells ECS which image to deploy. The pipeline has three stages: Source (pulls code from GitHub), Build (runs CodeBuild), and Deploy (updates ECS service with the new image using rolling deployment). Now, whenever you git push to the main branch, the entire build-and-deploy process runs automatically in about 5-7 minutes — no manual steps required.

1) CodeCommit — “Where your code lives” (Git repo)

What it is

AWS’s managed **Git repository** (like GitHub, GitLab).

Example

You create a repo:

- `proj2-app`

You push:

- `app.py` / `server.js`
- `Dockerfile`
- `taskdef.json` (ECS task definition template)
- `buildspec.yml` (instructions for CodeBuild)

What happens in real workflow

You do:

- `git push origin main`

That push becomes the trigger for the rest of the pipeline.

✅ So CodeCommit = **source code storage + trigger** ↓

2) CodeBuild — “Build + test + create artifacts”

What it is

A managed build server that:

- installs dependencies
- runs tests
- builds packages
- builds Docker images
- produces “artifacts” (outputs)

Example (ECS app)

When you push code, CodeBuild can:

Step A: run tests

- `pytest`
- `npm test`

Step B: build Docker image

- `docker build -t proj2-app .`

Step C: push to ECR

- `docker push <account>.dkr.ecr.us-east-1.amazonaws.com/proj2-app:<tag>`

Step D: output artifacts for deployment

For ECS deploy, CodeBuild often outputs:

- `imagedefinitions.json` (tells ECS which new image tag to use)
- or
- a rendered `taskdef.json`

✅ So CodeBuild = **compiler + tester + docker builder**

3) CodeDeploy — “Deployment automation” (where it’s used)

What it is

A deployment service that can deploy to:

- EC2 (classic)
- Lambda
- ECS (blue/green deployments)

Why your note says “we don’t use this”

For ECS, many setups deploy using:

- **CodePipeline’s ECS deploy action**
- or CodeDeploy if you specifically want **blue/green** deployment style

Example: Deploying to EC2 (classic use)

If you had an app on EC2 instances:

- CodeDeploy would:
 - copy new code
 - stop old version
 - start new version
 - run scripts (`appspec.yml`)

Example: Deploying to ECS with Blue/Green

If you want safer releases:

- CodeDeploy can run:
 - old tasks = “blue”
 - new tasks = “green”
 - shift traffic gradually using ALB
 - rollback if health checks fail

✅ So CodeDeploy = **deployment engine**, especially for EC2 and advanced ECS strategies

4) CodePipeline — “The conductor / orchestrator”

What it is

CodePipeline connects the whole CI/CD flow into stages.

It doesn't “build” or “deploy” by itself.

It **calls other services** in order.

Example pipeline for your ECS app

Stage 1: Source

- “Get latest code from CodeCommit”

Trigger: `git push`

Stage 2: Build (CodeBuild)

- Run tests
- Build Docker image
- Push to ECR
- Generate `imagedefinitions.json`

Stage 3: Deploy (ECS Deploy action OR CodeDeploy)

- Update ECS service to new image
- ECS starts new tasks
- ALB health checks
- Old tasks replaced (rolling update)

✅ So CodePipeline = **the workflow manager**

PHASE H: AUTO-SCALING WITH FARGATE SPOT

In Phase H, we implemented auto-scaling and cost optimization for our ECS service. We configured Fargate Spot as a capacity provider, which uses AWS's spare compute capacity at up to 70%

discount compared to regular Fargate — with the tradeoff that tasks can be interrupted with 2 minutes notice (we keep at least 1 task on regular Fargate for reliability). We set up

Application Auto Scaling with two target tracking policies: one that scales based on CPU utilization (target 70%) and another based on ALB request count (target 100 requests per target per

minute). When load increases, tasks scale out quickly (60-second cooldown); when load decreases, tasks scale in slowly (300-second cooldown) to avoid flapping. We also added scheduled

scaling to save costs: at 10 PM UTC the service scales to 0 tasks (complete shutdown), and at 8 AM UTC it scales back up to 1-4 tasks. The capacity provider strategy uses an 80/20 split — 80% Fargate Spot, 20% regular Fargate. Combined, these optimizations reduce compute costs by roughly 83% compared to running regular Fargate 24/7.

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

proj2-cluster

Fargate

Last updated 20 January 2026, 19:20 [UTC-6:00]

Actions

Create with Express Mode

Cluster overview

ARN: arn:aws:ecs:us-east-1:501894534533:cluster/proj2-cluster

Status: Active

CloudWatch monitoring: Default

Registered container instances: -

Services: Draining: -

Active: 1

Tasks: Pending: -

Running: -

Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Capacity providers (2)

info

Last updated 20 January 2026, 19:20 [UTC-6:00]

Manage tags

Update

Delete

Create

Filter capacity providers by property or value

Capacity provider	Scaling type	Provisioning model	Update status	Update status reason
Fargate	Fargate	-	-	-
Fargate Spot	Fargate Spot	-	-	-

Container instances (0)

info

Last updated 20 January 2026, 19:20 [UTC-6:00]

Register external instances

Actions

Filter container instances by property or value

Container instance	Instance type	Status	Statu...	Type	Instance ID	Capacit...	Availability zo...	Running tasks...	CPU available
--------------------	---------------	--------	----------	------	-------------	------------	--------------------	------------------	---------------

No container instances

No container instances to display.

To register Instances, use either [EC2 Auto Scaling group](#) or use [EC2 console](#)